1. Day 4

```
2. package Day4;
3.
4. public interface Movable {
5.     void moveUp();
6.     void moveDown();
7.     void moveLeft();
8.     void moveRight();
9. }
```

2.

```
package Day4;

public class MovableCircle implements Movable {
    int radius;
    MovablePoint center;

    public MovableCircle(int radius, MovablePoint center) {
        this.radius = radius;
        this.center = center;
    }

    @Override
    public void moveUp() { center.moveUp(); }
    @Override
    public void moveDown() { center.moveDown(); }
    @Override
    public void moveLeft() { center.moveLeft(); }
    @Override
    public void moveRight() { center.moveRight(); }

    @Override
    public String toString() {
```

```
        return "Circle radius=" + radius + " center=" + center;
    }
}

package Day4;

public class MovablePoint implements Movable {
    int x, y, xSpeed, ySpeed;

    public MovablePoint(int x, int y, int xSpeed, int ySpeed) {
        this.x = x;
        this.y = y;
        this.xSpeed = xSpeed;
        this.ySpeed = ySpeed;
    }

    @Override
    public void moveUp() { y -= ySpeed; }
    @Override
    public void moveDown() { y += ySpeed; }
    @Override
    public void moveLeft() { x -= xSpeed; }
    @Override
    public void moveRight() { x += xSpeed; }

    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }
}
```

```java
package Day4;

public class MovableRectangle implements Movable {
    MovablePoint topLeft;
    MovablePoint bottomRight;

    public MovableRectangle(MovablePoint topLeft, MovablePoint bottomRight) {
        if (topLeft.xSpeed != bottomRight.xSpeed || topLeft.ySpeed != bottomRight.ySpeed) {
            throw new IllegalArgumentException("Both points must have same speed!");
        }
        this.topLeft = topLeft;
        this.bottomRight = bottomRight;
    }

    @Override
    public void moveUp() {
        topLeft.moveUp();
        bottomRight.moveUp();
    }
    @Override
    public void moveDown() {
        topLeft.moveDown();
        bottomRight.moveDown();
    }
    @Override
    public void moveLeft() {
        topLeft.moveLeft();
        bottomRight.moveLeft();
```

```
    }
    @Override
    public void moveRight() {
        topLeft.moveRight();
        bottomRight.moveRight();
    }

    @Override
    public String toString() {
        return "Rectangle topLeft=" + topLeft + " bottomRight=" + bottomRight;
    }
}


package Day4;

public interface Polygon {
    double getArea();

    // Default method to compute perimeter
    default double getPerimeter(int... sides) {
        double sum = 0;
        for (int side : sides) {
            sum += side;
        }
        return sum;
    }

    // Static helper method
    static String shapeInfo() {
```

```
        return "Polygons are 2D shapes with straight sides.";
    }
}
```

```
package Day4;

public class Rectangle implements Polygon {
    double length, width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    public double getArea() {
        return length * width;
    }
}
```

```
package Day4;

public class Triangle implements Polygon {
    double base, height;

    public Triangle(double base, double height) {
        this.base = base;
        this.height = height;
```

```
    }

    @Override
    public double getArea() {
        return 0.5 * base * height;
    }
}


package Day4;

public class Main {
    public static void main(String[] args) {
        // MovablePoint
        MovablePoint p1 = new MovablePoint(0, 0, 2, 2);
        System.out.println("Point: " + p1);
        p1.moveRight();
        p1.moveDown();
        System.out.println("After moving: " + p1);

        // MovableCircle
        MovableCircle circle = new MovableCircle(5, new MovablePoint(1, 1, 1, 1));
        System.out.println(circle);
        circle.moveUp();
        circle.moveLeft();
        System.out.println("After moving: " + circle);

        // MovableRectangle
        MovableRectangle rect = new MovableRectangle(
                new MovablePoint(0, 0, 1, 1),
```

```
            new MovablePoint(4, 4, 1, 1)
        );
        System.out.println(rect);
        rect.moveDown();
        rect.moveRight();
        System.out.println("After moving: " + rect);
    }
}
```

```
package Day4;

public class Main2 {
    public static void main(String[] args) {
        // Rectangle
        Rectangle r = new Rectangle(10, 5);
        System.out.println("Rectangle area: " + r.getArea());
        System.out.println("Rectangle perimeter: " + r.getPerimeter(10, 5, 10, 5));

        // Triangle
        Triangle t = new Triangle(6, 4);
        System.out.println("Triangle area: " + t.getArea());
        System.out.println("Triangle perimeter: " + t.getPerimeter(6, 4, 5));

        // Static method call
        System.out.println(Polygon.shapeInfo());
    }
}
```

```
package Day4;

public class MainBoth {
    public static void main(String[] args) {
        System.out.println("=== Movable Shapes Simulation ===");

        // MovablePoint
        MovablePoint p1 = new MovablePoint(0, 0, 2, 2);
        System.out.println("Point: " + p1);
        p1.moveRight();
        p1.moveDown();
        System.out.println("After moving: " + p1);

        // MovableCircle
        MovableCircle circle = new MovableCircle(5, new MovablePoint(1, 1, 1, 1));
        System.out.println(circle);
        circle.moveUp();
        circle.moveLeft();
        System.out.println("After moving: " + circle);

        // MovableRectangle
        MovableRectangle rect = new MovableRectangle(
                new MovablePoint(0, 0, 1, 1),
                new MovablePoint(4, 4, 1, 1)
        );
        System.out.println(rect);
        rect.moveDown();
        rect.moveRight();
        System.out.println("After moving: " + rect);
        System.out.println("\n=== Polygon Interface  ===");
```

```
        Rectangle r = new Rectangle(10, 5);
        Triangle t = new Triangle(6, 4);
        System.out.println("Rectangle area: " + r.getArea());
        System.out.println("Rectangle perimeter: " + r.getPerimeter(10, 5, 10, 5));
        System.out.println("Triangle area: " + t.getArea());
        System.out.println("Triangle perimeter: " + t.getPerimeter(6, 4, 5));
        System.out.println(Polygon.shapeInfo());
    }
}
```